



4EYES Labs Audit

Lithos: Mining (Part 3)

Auditor: Luca D'Angelo

Date: 2026-09-20

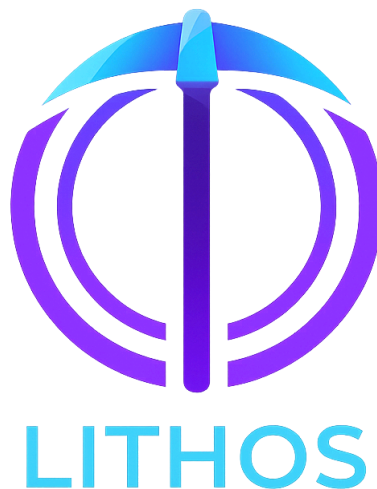


Table of Contents

Table of Contents	2
Disclaimer	3
Audit Summary	4
Contracts	5
Collateral Enforcer	6
Collateral Mainnet	10
Holding Logic	17
Holding Guard	21
Evaluation	25
Payout	30
Appendix	36
Issue Guide	37
About Us	37

Disclaimer

This audit is provided as-is at the time of its completion and might not reflect the most up-to-date state of any smart-contracts deployed, or any design document, or any research paper referenced herein.

Audit Summary

Protocol	Lithos: Mining (Part 3)
Repository	https://github.com/Lithos-Protocol/Lithos-Client
Commit	c87a1e431a66ebd4d22e4842fe920b90220420b4

Contract	File Name	Severity	Status
Collateral Enforcer	Collateral_Enforcer.ergo	GOOD	RESOLVED
Collateral Mainnet	Collateral_Mainnet.ergo	GOOD	RESOLVED
Holding Logic	Holding_Logic.ergo	GOOD	RESOLVED
Holding Guard	Holding_Guard.ergo	GOOD	RESOLVED
Evaluation	Evaluation.ergo	GOOD	RESOLVED
Payout	Payout.ergo	GOOD	RESOLVED

Contracts

Collateral Enforcer

Description

- Contract checking that a collateral box is created properly from a queue box.
- The contract is executed as a context-variable by the Emission Guard box.

Box Contents

Registers

- The same as the emission guard.

Tokens

- The same as the emission guard.

Context Variables

- Index 0 [Byte]: References CTX_OP of the emission guard input box at index 0 in the transaction inputs, used for choosing the correct spending path to take:
 - 0: Join - Long a lender's principal and permit into a queue box added to the tail of the queue.
 - 1: Activate - Convert the head queue box into a collateral box.
 - 2: Clear - Remove a head queue box from the queue if the corresponding lender already exists in the lender set.
- Index 2 [GroupElement]: References CTX_GE of the emission guard input box at index 0 in the transaction inputs, representing the group element of the lender's p2pk.

Compile-Time Constants

- CONST_LIT_ID [Coll[Byte]]: Token id of the LIT token.

Contract Analysis

- If CTX_OP is 0 (Join) then the following is checked:
 - L70 defines queueOut as the output queue box.
 - L73 gets the CTX_GE GroupElement context-variable at index 2 from the input 0 emission guard box.
 - L75 defines queueGrown checking that:
 - The nextHead position equals the queueHead position from the input emission box. A queue is FIFO, so when an element gets added to the queue, the tail gets longer but the head remains the same.
 - The nextTail position equals the queueTail position from the input emission box incremented by 1.
 - L81-L84 accesses the permitParams from R4 of the emissionConfig data-input.
 - 0: The permit fee floor.
 - 1: The permit fee ceiling.
 - 2: The permit fee slope.
 - L85 defines backlog as the difference between queueTail and queueHead.
 - This represents how many outstanding queue boxes there are that have not yet become collateral boxes.
 - This value is used to determine how LIT permit tokens are required for providing collateral.
 - L86 defines scaled as the value of the permitFloor variable plus the product of the permitSlope and backlog variables.
 - This variable defines how many LIT permit tokens a lender must provide in order to provide their ERG as collateral.
 - It follows a linear scale, such that a larger queue backlog leads to requiring more LIT permit tokens.
 - Thus, there is an incentive for lenders to join the queue, i.e. provide collateral, as soon as possible, since it will require them to provide fewer LIT permit tokens (lenders must purchase LIT tokens to provide as permits).
 - L87-L92 defines permitRequired checking that:
 - If scaled is greater than or equal to the permitCeil variable, representing the maximum amount of permits a lender is required to provide, then permitCeil is used.
 - Otherwise, scaled is used.
 - L93 defines finderFee as the difference between the Long fee value stored in R4 of the queue box and the CONST_DUST_BUDGET variable.
 - L95 defines queueLIT as the LIT tokens in the output queue box.
 - L96 defines permitPosted checking that the amount of LIT permit tokens in the output queue box equals the permitRequired amount.
 - L97 defines permitValid checking that the token id of the LIT permit tokens in the output queue box equals the CONST_LIT_ID compile-time constant.
 - L98 defines queueProp as the SigmaProp in R5 of the output queue box.
 - L99-L121 defines queueBoxValid checking that:

4EYES Labs Audit - Lithos: Mining (Part 3)

- R7 of the output queue box is a Long equal to the queueTail value of the input emission box.
 - Incrementing the tail tells us how many queue boxes have been created.
 - Incrementing the head tells us how many queue boxes have been consumed to create collateral boxes.
 - The difference between tail and head tells us the size of the active queue.
- The proposition bytes of the SigmaProp stored in R5 of the output queue box equals the SigmaProp proposition bytes of the CTX_GE GroupElement.
 - This ensures the lender information is propagated throughout the protocol.
- The CTX_GE GroupElement is not the identity element.
- The finderFee is greater than or equal to 0.
- The ERG value in the output queue box is at least the CONST_DUST_BUDGET plus the CONST_BLOCK_REWARD minus the CONST_FEE_CAP plus the feeValue stored in R4 of the output queue box plus the multiple of finderFee and CONST_POOL_MULTIPLE.
 - The lender provides collateral at least equal to the Ergo blockchain block reward minus the maximum possible fee they can receive from the protocol (3%).
 - The lender can also include a tip to get their live collateral box chosen sooner, but doing so requires providing a 5x multiple to the pool as well.
 - Extra ERG is also needed for creating additional boxes further in the protocol.
- R4 of the output queue box is a Long equal to the CONST_DUST_BUDGET value for creating extra boxes created in the protocol.
- permitPosted is true.
- permitValid is true.
- R6 of the output queue box is a Byte equal to 0. It represents an extension flag.
- The output queue box R8 is undefined to enforce some maximum size on the box and prevent unnecessary bloat.
 - L122 checks that queueGrown and queueBoxValid are true.
- If CTX_OP is 1 (Activate) then the following is checked:
 - L125 defines queueIn as the second input box at index 1.
 - L126-L132 defines queueAdvanced checking that:
 - R7 of the input queue box corresponds to the queueHead Long value in the input emission box.

4EYES Labs Audit - Lithos: Mining (Part 3)

- This makes sure that the queue box being processed is the correct one in the queue order, i.e. the head pointer has reached the current queue box.
 - nextHead in the output emission box is incremented by 1, indicating that the current queue box has been processed.
 - nextTail in the output emission box is equal to the queueTail value in the input emission box.
- L133 defines collatBox as the output collateral box at index 1.
- L135 defines collatConstrained checking that R9 of the collateral box is undefined, limiting the box size to prevent NISP bloat from spam.
- L136 checks that queueAdvanced and collatConstrained are true.
- If CTX_OP is 1 (Clear) then the following is checked:
 - L139 defines queueIn as the second input box at index 1.
 - L140-L146 defines queueCleaned checking that:
 - R7 of the input queue box corresponds to the queueHead Long value in the input emission box.
 - This makes sure that the queue box being processed is the correct one in the queue order, i.e. the head pointer has reached the current queue box.
 - So a clear/refund must still occur in the correct queue order.
 - nextHead in the output emission box is incremented by 1, indicating that the current queue box has been processed.
 - nextTail in the output emission box is equal to the queueTail value in the input emission box.
 - L147 checks that queueCleaned is true.
- Otherwise, evaluate to false.

Transaction Diagrams

- None, same as Part 1.

Review Comments

- Logic for the addition of a finder fee has been added to this new version of the contract. It functions as a tip the lender can provide for their live collateral box to be processed sooner. However, the lender must provide a 5x multiple to the pool as well.

Collateral Mainnet

Description

- The contract containing the lender's provided collateral used to pay the Lithos pool for finding a block.
- The collateral box can only be spent by a miner and it removes the lender from the emission box's active lender set.

Box Contents

Registers

- R4 [Long]: Fee value as a Long taken from the block reward to fund other boxes and any tip for processing a collateral box sooner.
- R5 [SigmaProp]: Lender PK as a SigmaProp representing the lender that will receive the block's coinbase.
- R6 [(Long, Coll[(Coll[Byte], Long)])]: The LIT rewards as a tuple of a Long value representing the LIT emission for the block and a collection of tuples representing the founder addresses and their respective share allocation of the LIT emissions amount. These amounts were calculated by the emissions contract when creating the collateral box.
- R7 [Byte]: The extension flag as a Byte representing which collateral extension script must also be satisfied. Initially, this is set to 0 since there is no extension script at mainnet launch.
- R8 [Coll[Byte]]: The hashed proposition bytes of the rollup-holding script as a Coll[Byte]. This is defined in the emission config box so it can be changed in the future, however, the collateral box keeps the same hash that was used to create it.

Tokens

- Index 0: The collateral token.
- Index 1: The LIT permit tokens + LIT emission tokens.

Context Variables

- Index 0 [Coll[Byte]]: CTX_EXT_SCRIPT representing the serialized value bytes of the collateral extension script, read if there is an extension flag set.

Compile-Time Constants

- CONST_GATE_HASH [Coll[Byte]]: the hashed proposition bytes of the emission gate contract, which is used to represent either a queue box or a proof-of-spend box.
- CONST_EMCONFIG_NFT [Coll[Byte]]: the token id of the emission config NFT uniquely identifying the box.
- CONST_LIT_ID [Coll[Byte]]: the token id of the LIT token.

Contract Analysis

- L40 defines `CONST_FINDER_DIVISOR` as a Long equal to 25 used to give a fixed 4% of the LIT emission for the person who builds the genesis transaction.
- L44 defines `holdingBox` as the first output at index 0.
- L63-L68 gets the LIT block reward and private reward amounts from the set values in R6.
- L69 computes `totalLITRewards` as the sum of the lit block reward and the lit private reward.
- L72 defines `permitChange` as the difference between the amount of LIT tokens in the collateral box and the `totalLITRewards` calculated.
 - This difference represents the lender's LIT permit tokens, which will be returned to the lender once the collateral box is spent.
- L74-L87 defines `foundersPaid` checking that:
 - If the LIT private rewards are greater than 0, then:
 - For each founder in the founder map there exists a corresponding output such that:
 - There is a token at index 0 or fallback to a default value.
 - The blake2b256 hash of the box proposition bytes equals the founder proposition bytes.
 - The box value is greater than or equal to 1_000_000 nanoErg.
 - The token at index 0 has the `CONST_LIT_ID` compile-time constant as a token id and the amount is equal to the corresponding value from the founder map tuple element.
 - Otherwise, return true.
- L89-L100 defines `permitChangeReturned` checking that:
 - If `permitChange` is greater than 0, then:
 - There exists an output such that:
 - There is a token at index 0 or fallback to a default value.
 - The box proposition bytes equals the proposition bytes of the lender's `SigmaProp` stored in the register of the collateral box.
 - The box value is greater than or equal to 1_000_000 nanoErg.
 - Even though the min nanoErg per byte is a parameter that can hypothetically change in the future, if it is lowered for example, `feeValue` will increase and more ERG will be provided.
 - The token at index 0 has the `CONST_LIT_ID` compile-time constant as a token id and the amount is equal to `permitChange`.
 - Otherwise, return true.
- L113-L120 define `proofOfSpendMade` checking that there exists an output such that:
 - There is a token at index 0 or fallback to a default value.
 - The blake2b256 hash of the proposition bytes of the box equals the `CONST_GATE_HASH` compile-time constant, checking that a proof-of-spend box is created.

4EYES Labs Audit - Lithos: Mining (Part 3)

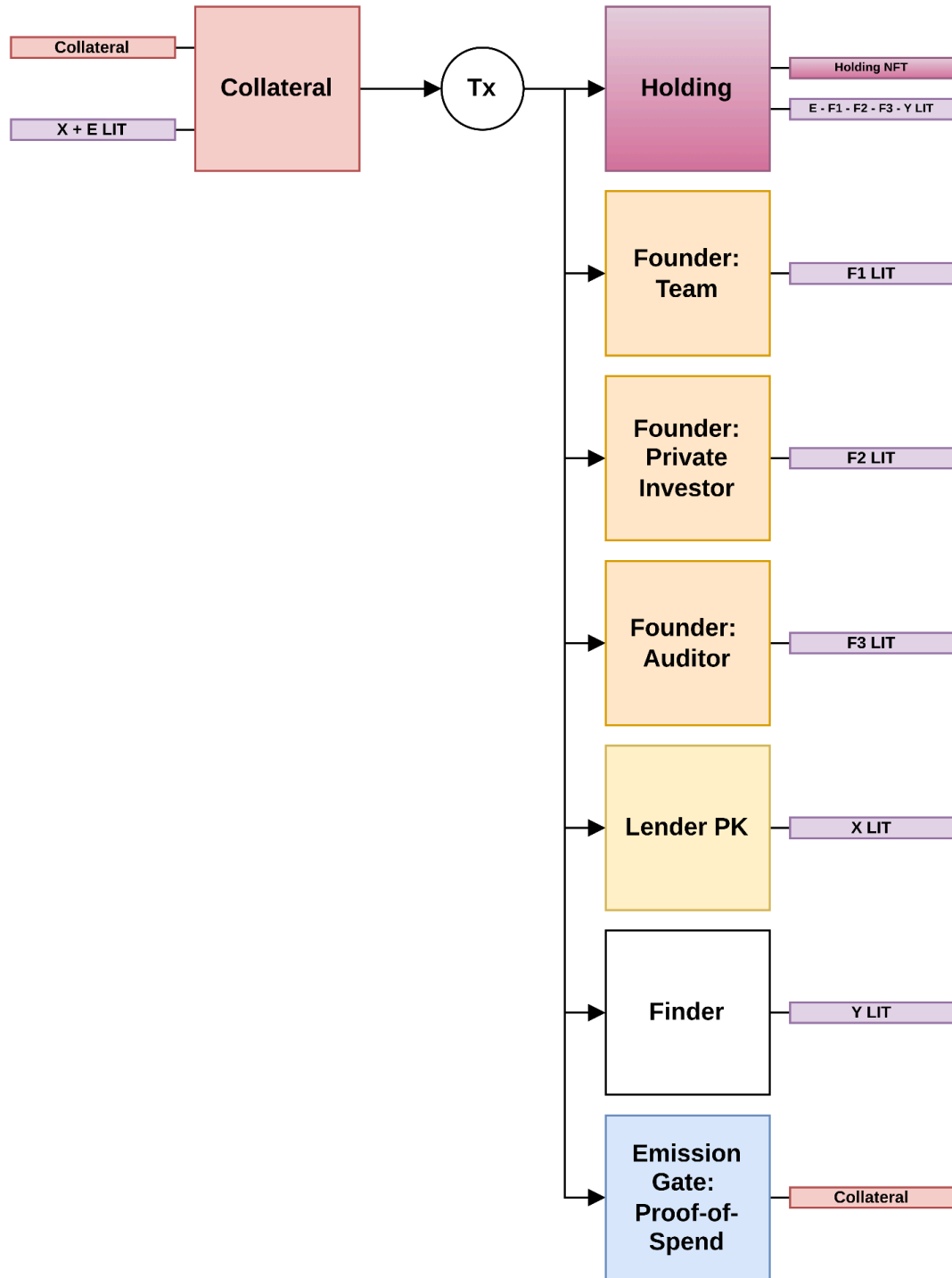
- The token id equals the collateral token id in the collateral box and the amount is 1.
- The creation-height of the proof-of-spend box equals the current height of the block being created.
 - Following the LIT emission contract, a lender can only be removed from the active set once the creation-height of the proof-of-spend box is old enough.
- R4 of the output proof-of-spend box is a Coll[Byte] equal to the blake2b256 hash of the lender's SigmaProp proposition bytes.
- L122-L141 defines extensionActivated checking that:
 - If the extensionFlag in the collateral box is not equal to 0, then:
 - Get the extension script CTX_EXT_SCRIPT from the transaction context at index 0, as a Coll[Byte].
 - The emission config is expected as a data-input at index 0.
 - L132 defines authenticConfig checking that the emission config box contains the CONST_EMCONFIG_NFT compile-time constant at index 0.
 - The extension script hash is obtained from the emission config at register R6.
 - L135 defines authenticExtension checking that the blake2b256 of the CTX_EXT_SCRIPT context variables equals the extension script hash from the emission config register R6.
 - L137 checks that authenticConfig and authenticExtension are true and that the CTX_EXT_SCRIPT executed from the transaction context evaluates to true.
 - Otherwise, evaluate sigamProp(true).
- L149 defines finderLIT as the quotient of the LIT block rewards and the CONST_FINDER_DIVISOR. This ensures that whoever creates the genesis transaction gets 4% of the LIT block reward.
- L150 defines poolLIT as the LIT block reward minus the finderLIT, this is the LIT given to miners of the mining pool.
- L152-L159 defines tokensTransferred checking that:
 - If the poolLIT is greater than 0, then:
 - The holding box contains a poolLIT amount of LIT tokens at index 1.
 - Otherwise, return true.
- L161 defines rollupState as the Coll[Long] in R7 of the output holding box, containing the following parameters:
 - Index 0: period start height.
 - Index 1: block height.
 - Index 2: bond total.
- L162 defines holdingNFT checking that the output holding box contains an NFT at index 0 with the id of the collateral input box.
- L163 defines numNFT counting the cumulative amount of tokens in the outputs whose id is the box id of the input collateral box.

4EYES Labs Audit - Lithos: Mining (Part 3)

- L171-L181 defines validAvlTree checking that R4 of the holding box contains the AvlTree where:
 - The digest is the empty digest.
 - All operations (insert, update, and remove) are allowed.
 - The key length is 32.
 - Values can be of different lengths (hash of miner bytes are constant size, but NISP proofs may be different sizes).
- L183-L204 defines holdingValid checking that:
 - The holding box value is at least the value of the collateral box minus the feeValue used to create any other box in the outputs.
 - The blake2b256 hash of the proposition bytes equals the rollupHoldingHash.
 - tokensTransferred is true.
 - validAvlTree is true.
 - R5 of the holding box is an Int equal to 0.
 - R6 of the holding box is a BigInt equal to 0.
 - R7 of the holding box is a Coll[Long] where:
 - The size of the collection is 3.
 - Index 0 and 1 are set to the current HEIGHT, and index 2 is set to 0.
 - R8 of the holding box is a Coll[Byte] with size equal to 32 representing the hash of the proposition bytes of the miner.
 - holdingNFT is true.
 - numNFT equals 1.
- L209 defines lenderIsBlockMiner checking that the lenderPk SigmaProp proposition bytes equals the SigmaProp proposition bytes of the minerPk from the block preHeader.
 - This ensures that the lender is the miner at the protocol level, ensuring they receive the full ERG block reward.
- L210 defines onlyOne checking that the first input at index 0 is the current collateral box, ensuring that no other box can exist at input index 0.
- L212-L221 defines validLithosBlock checking that:
 - holdingValid, permitChangeReturned, foundersPaid, proofOfSpendMade, lenderIsBlockMiner, and onlyOne are true.
- L223 checks that validLithosBlock is true and that extensionsActivated is true.

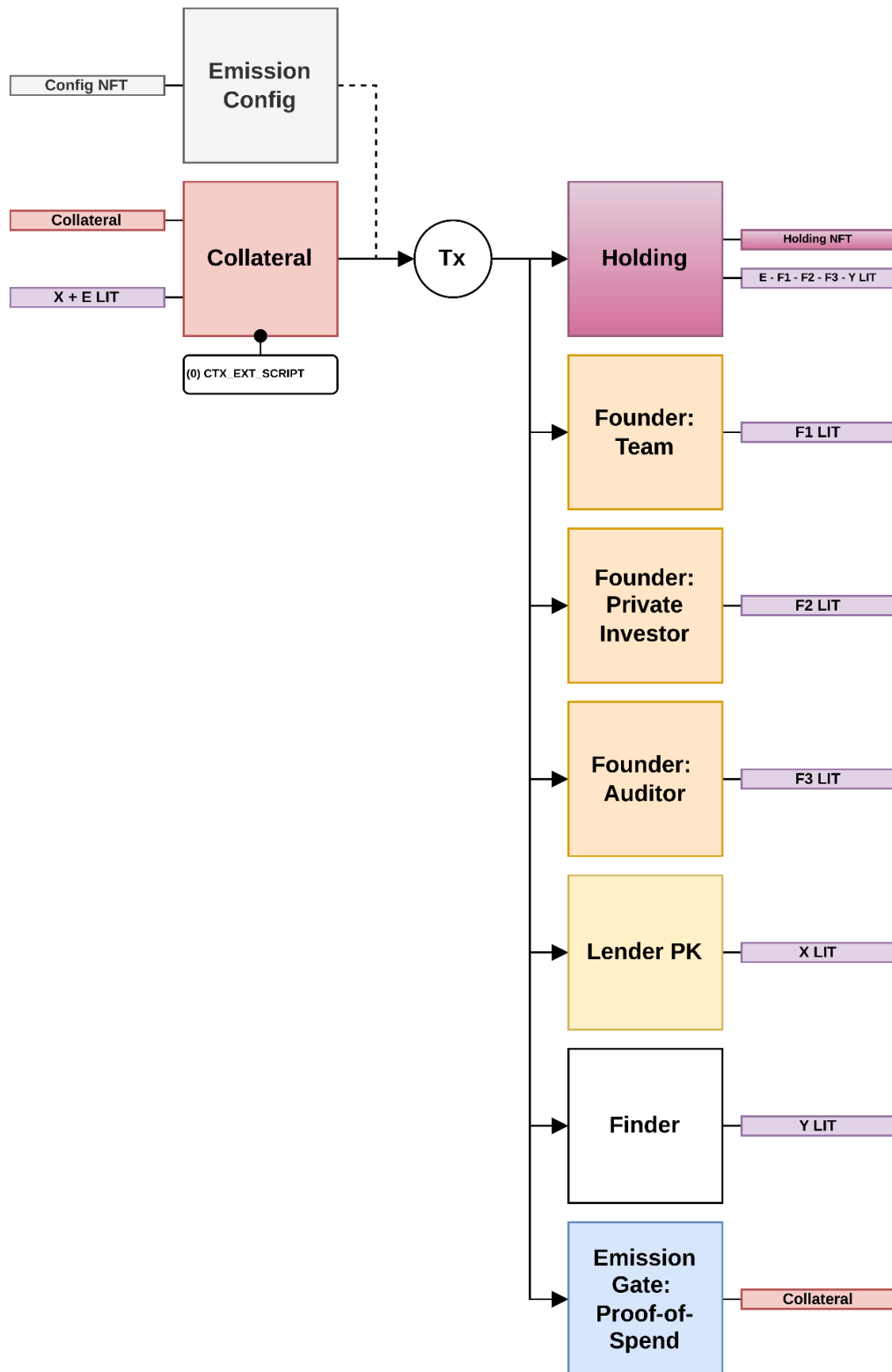
Transaction Diagrams

Genesis: No Extension



4EYES Labs Audit - Lithos: Mining (Part 3)

Genesis: With Extension



4EYES Labs Audit - Lithos: Mining (Part 3)

Review Comments

- The holding box now contains a holding NFT.
- The register R7 of the holding box has been modified to be a collection of Long values representing the rollup state. Part of this state is the bond requirement for NISPs, which will be explored further with the holding logic contract.

Holding Logic

Description

- This contract holds the ERG block reward provided by the lender's collateral and the LIT tokens to be emitted to the pool that was provided by the emission box.
- During the submission period, miners provide their NISP to be included in the AVL tree inside the Holding Guard box.
- After the submission period ends, no more NISPs can be provided by miners. This contract ensures the Holding Guard box becomes the evaluation box, where fraud proofs will be used to check that the submitted NISPs were not created fraudulently.

Box Contents

Registers

- Same as the Holding Guard.

Tokens

- Same as the Holding Guard.

Context Variables

- Index 1 [(Coll[Byte], Coll[Byte])]: CTX_NISP_DATA is the data to be inserted into the NISP tree, a pair of the hashed miner proposition bytes and the corresponding NISP.
- Index 2 [Coll[Byte]]: CTX_NISP_PROOF is the AVL Tree insertion proof.
- Index 3 [Byte]: CTX_OP determines the spending path to take.
 - 0: Submit (submission phase).
 - 1: Transform (convert to evaluation box).

Compile-Time Constants

- CONST_PERIOD_LENGTH [Int]: The length of the NISP submission period, used for determining the appropriate spending path in the holding contract.
- CONST_EVAL_PROPBYTES [Coll[Byte]]: The hashed proposition bytes of the evaluation contract.
- CONST_MIN_ENTRY_BOUND [Long]: Floor on the refundable bond a submission is required to provide.
- CONST_BOND_DIVISOR [Long]: Score units per nanoERG of bond.

Contract Analysis

- L92 defines the `bondForScore` function taking a `bondScore` Long input and doing the following:
 - L93 defines `bondProportional` as the quotient of `bondScore` and `CONST_BOND_DIVISOR`, representing the amount of ERG required for the bond based on its score.
 - L93 checks that:
 - If the `bondProportional` value is greater than `CONST_MIN_ENTRY_BOND` then use the `bondProportional` value.
 - Otherwise, use the `CONST_MIN_ENTRY_BOND` value.
- L98 if `CTX_OP` is 0 (submission) then:
 - If the size of the NISP is greater or equal to `CONST_NISP_MIN` then the following is checked:
 - L100 defines `inSubmissionPhase` checking that:
 - The height of the current block is less than the submission phase period start height plus the `CONST_PERIOD_LENGTH`.
 - The height of the current block does not equal the submission phase blockheight.
 - L101 defines `validUTXO` checking that the output holding box has the same proposition bytes as the input holding box.
 - L103 defines `scoreBytes` as the first 8 bytes of the provided NISP context variable.
 - L104 defines `score` as the Long representation of `scoreBytes`.
 - L111 defines `validNISP` checking that:
 - The score is positive.
 - The total score in R6 of the output holding box is the total score of the input holding box plus the score of the provided NISP.
 - The provided NISP size is less than `CONST_NISP_MAX`.
 - L116 defines `entryBond` as the result from the `bondForScore` method applied to the NISP score input.
 - L117 defines `bondCollected` checking that all of the following are true:
 - The output holding box value equals the sum of the input holding box value plus the `entryBond` value.
 - The bond total of the output holding box equals the input bond total plus the calculated `entryBond` value (a tally of the accumulated bond is stored as an entry in the rollup state collection).
 - L124-L132 defines `treeUpdated` checking that:
 - L127 defines `NISPHash` as the blake2b256 hash of the entire NISP.
 - L129 defines `NISPHashKV` as the key-value pair of the following:
 - Key: provided hashed miner proposition bytes from the `CTX_NISP_DATA` context-variable.

4EYES Labs Audit - Lithos: Mining (Part 3)

- Value: The NISPHash appended to the first 8 bytes of the NISP (the score bytes).
 - L131 checks that the output tree digest equals the inserted tree digest.
 - L133 defines onlyOne checking that the box at input 0 has the id of the current box.
 - L134 defines incrementedMiners checking that the miner count increased by 1.
 - L135 defines samePeriod checking that the period start height does not change in the output holding box.
 - L136 defines sameBlock checking that the creation-height of the initial holding box is preserved in the output holding box.
 - L137 defines tokensConserved checking that the tokens in the output holding box are the same as the tokens in the input holding box.
 - L138 defines statements checking that all of the following are true:
 - inSubmissionPhase, onlyOne, validUTXO, stateSized, bondCollected, validNISP, treeUpdated, incrementedMiners, samePeriod, sameBlock, tokensConserved.
 - L154 checks that statements evaluates to true.
 - Otherwise, evaluate to false.
- L159 if CTX_OP is 1 (transform) then:
 - If the current-height is the creation-height of the initial holding box (i.e. the genesis block), then:
 - L165 defines valueAdded where all properties of the output holding box are preserved except for the box value which must be strictly greater than the input holding box value.
 - This transaction can happen if the initial holding box is chained in the same block.
 - Otherwise, the following is checked for a valid transformation to the evaluation contract.
 - L182 defines conservedValued where all properties of the output evaluation box are kept the same as the input holding box other than the output period start height being less than or equal to the current blockheight and greater than or equal to the submission period end blockheight.
 - L198 defines isEvaluationContract checking that the output evaluation box's hashed proposition bytes equal the CONST_EVAL_PROPBYTES compile-time constant.
 - L199 checks that isEvaluationContract and conservedValues are true.
- Otherwise, evaluate to false.

4EYES Labs Audit - Lithos: Mining (Part 3)

Transaction Diagrams

- The same as the holding guard script.

Review Comments

- Submitting a NISP now requires a bond to be provided by the miner, which will be returned if the NISP is not fraudulent.

Holding Guard

Description

- Executes the holding logic script from a context-extension variable.
- Requires the miner to “sign” the transaction, either as a P2PK or P2S.

Box Contents

Registers

- R4 [AvlTree]: The NISP tree, initially an empty AVL tree mapping a miner’s hashed proposition bytes to their corresponding NISP.
- R5 [Int]: Total miners in a given period, representing miners who have posted a NISP.
- R6 [BigInt]: The total share score, which increases for every NISP posted. The Share Score of a NISP is the first 8 bytes of the NISP. Individual Share Scores have type Long.
- R7 [Coll[Long]]: Collection of values representing the rollup state.
 - Index 0: Period start.
 - Index 1: Block height (creation/mining height of the initial holding box).
 - Index 2: Bond total (refundable ERG this rollup holds on behalf of NISP submitters).
- R8: [Coll[Byte]]: Hashed proposition bytes of the miner.
- R9: Reserved for extensions.

Tokens

- Index 0: The holding box NFT that uniquely identifies this box.
- Index 1: LIT tokens to be emitted to miners of the pool.

Context Variables

- Index 0 [Coll[Byte]]: CTX_SIGNER is the value bytes of the signer of the transaction used to authenticate submission of the NISPs.
- Index 1 [(Coll[Byte], Coll[Byte])]: CTX_NIS_DATA is the data to be inserted into the NISP tree, a pair of the hashed miner proposition bytes and the corresponding NISP.
- Index 2 [Coll[Byte]]: CTX_NISP_PROOF is the AVL Tree insertion proof.
- Index 64 [Coll[Byte]]: CTX_LOGIC representing the serialized value bytes of the holding logic contract.

Compile-Time Constants

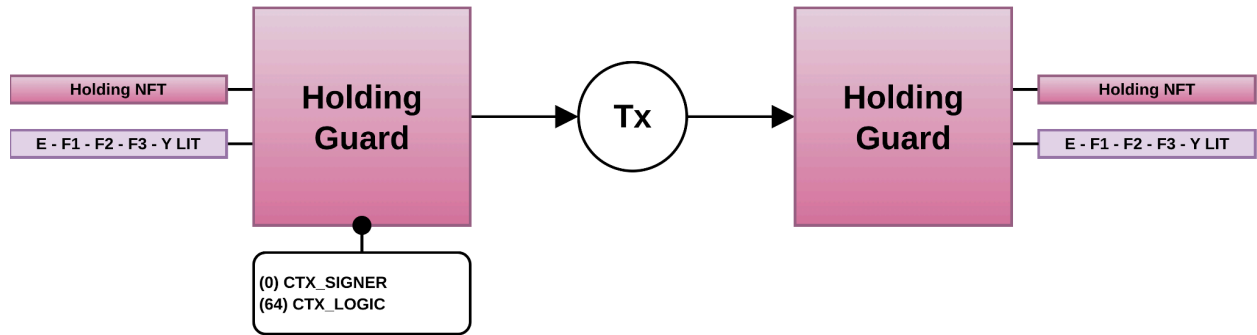
- CONST_HOLDING_LOGIC_HASH [Coll[Byte]]: The blake2b256 hash of the serialized value bytes of the holding logic contract.

Contract Analysis

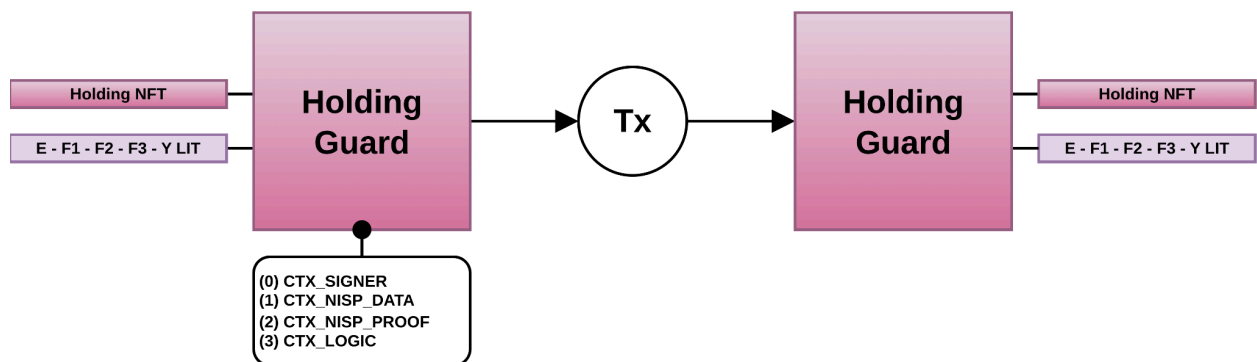
- L46 defines CTX_OP as the context-variable obtained at index 3.
- L47 defines CTX_LOGIC as the context-variable obtained at index 64.
- L49 defines authenticLogic checking that the blake2b256 hash of the CTX_LOGIC context-variable equals the CONST_HOLDING_LOGIC_HASH compile-time constant.
- L52 if CTX_OP is 0 (submission) then:
 - L53 defines CTX_NISP_DATA as the context-variable obtained at index 1.
 - L54 defines signerProp as the SigmaProp obtained from executing the serialized value bytes of the context-variable at index 0, representing the miner, which could be a P2PK or P2S address.
 - L58 defines authenticMiner checking that the blake2b256 of the proposition bytes of singerProp SigmaProp equals the provided hash in the CTX_NISP_DATA context-variable.
 - L60 checks that authenticLogic and authenticMiner are true while signerProp and the holding logic context-variable at index 64 evaluate to true.
- L61 if CTX_OP is 1 (transform) then:
 - L64 checks that authenticLogic is true and that the holding logic context-variable at index 64 evaluates to true.
- Otherwise, evaluate to false.

Transaction Diagrams

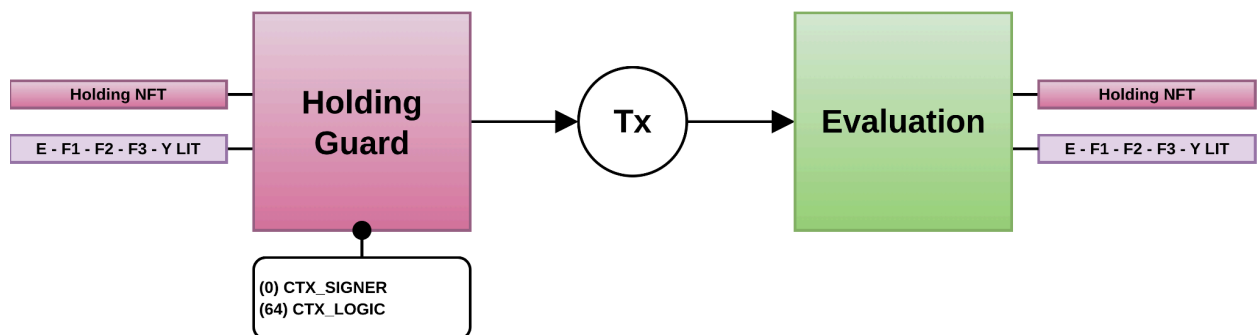
Holding: Add ERG (Genesis Block)



Holding: NISP Submission



Holding: Convert to Evaluation



4EYES Labs Audit - Lithos: Mining (Part 3)

Review Comments

- Unlike the previous version, there is now a guard script that executes the main holding logic from a context-variable instead.

Evaluation

Description

- During the evaluation phase, submitted NISPs are checked for fraud where specific fraud-proof logic is contained in corresponding fraud-proof contracts.
- The fraud-proof evaluation is optimistic assuming that miners provide valid NISPs and an evaluation transaction only occurs if someone finds valid fraud.

Box Contents

Registers

- R4 [AvlTree]: The NISP tree, initially an empty AVL tree mapping a miner's hashed proposition bytes to their corresponding NISP.
- R5 [Int]: Total miners in a given period, representing miners who have posted a NISP.
- R6 [BigInt]: The total share score, which increases for every NISP posted. The Share Score of a NISP is the first 8 bytes of the NISP. Individual Share Scores have type Long.
- R7 [Coll[Long]]: Collection of values representing the rollout state.
 - Index 0: Period start.
 - Index 1: Block height (creation/mining height of the initial holding box).
 - Index 2: Bond total (refundable ERG this rollout holds on behalf of NISP submitters).
- R8: Reserved for extensions.
- R9: Reserved for extensions.

Tokens

- Index 0: The holding box NFT that uniquely identifies this box.
- Index 1: LIT tokens to be emitted to miners of the pool.

Context Variables

- Index 0 [Coll[Byte]]: The fraud proof script.

Compile-Time Constants

- CONST_PERIOD_LENGTH [Int]: The length of the evaluation period, used for determining the appropriate spending path in the holding contract.
- CONST_PAYOUT_PROPBYTES [Coll[Byte]]: The hashed proposition bytes of the evaluation contract.
- CONST_FP_ID [Coll[Byte]]: Singleton token id for the box holding the valid fraud-proofs.
- CONST_MIN_ENTRY_BOUND [Long]: Floor on the refundable bond a submission is required to provide.
- CONST_BOND_DIVISOR [Long]: Score units per nanoERG of bond.

Contract Analysis

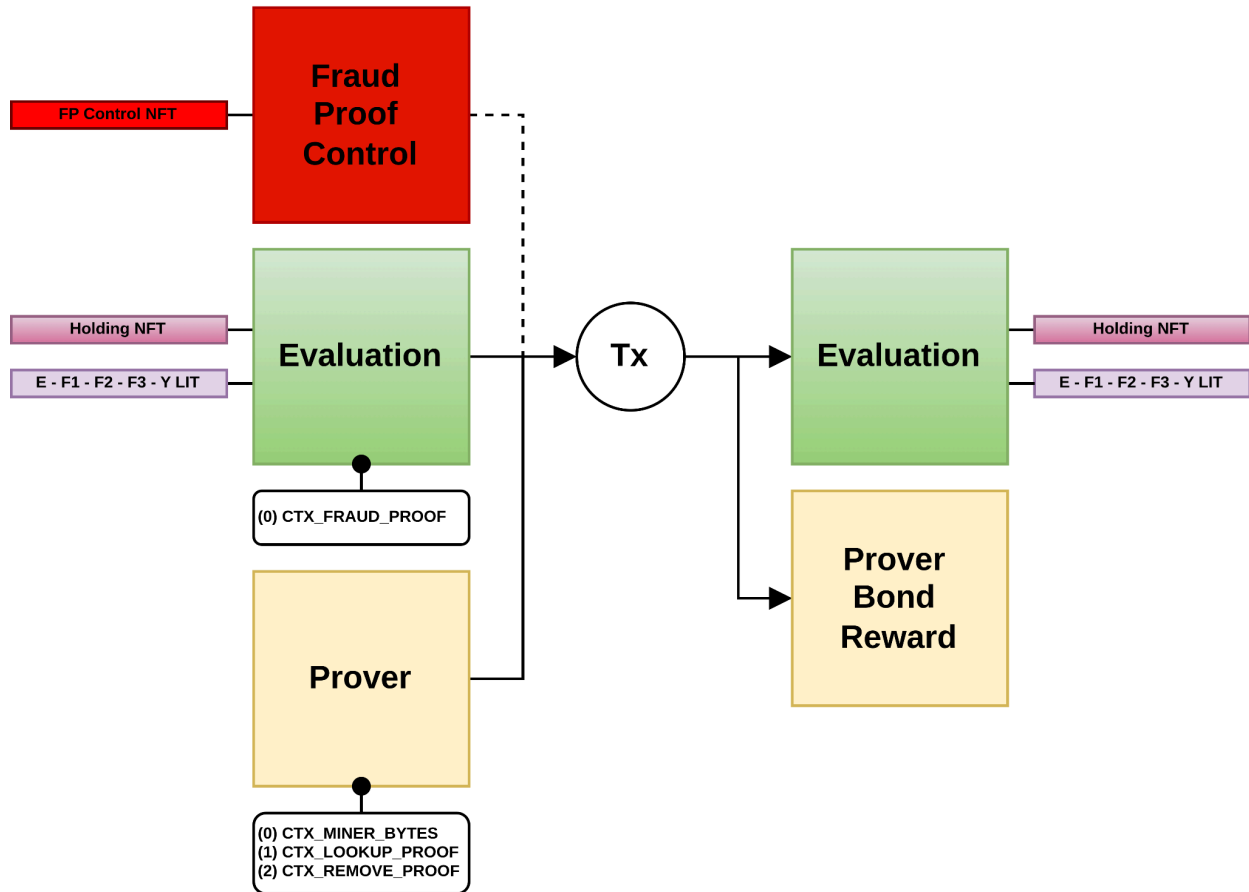
- L70 defines the `bondForScore` function taking a `bondScore` Long input and doing the following:
 - L93 defines `bondProportional` as the quotient of `bondScore` and `CONST_BOND_DIVISOR`, representing the amount of ERG required for the bond based on its score.
 - L93 checks that:
 - If the `bondProportional` value is greater than `CONST_MIN_ENTRY_BOND` then use the `bondProportional` value.
 - Otherwise, use the `CONST_MIN_ENTRY_BOND` value.
- L75 defines `onlyOne` checking that the evaluation box is the first input in the transaction at index 0.
- L80 if the current blockheight is within the evaluation phase period then:
 - L81 defines `fpData` as the first data-input in the transaction.
 - This box contains the hashed proposition bytes of the valid fraud-proof contracts that can be spent with the evaluation box.
 - L82 defines `authenticFPData` checking that the data-input contains the `CONST_FP_ID` singleton token at index 0.
 - L83 defines `script` getting the fraud-proof script bytes as a context-variable at index 0.
 - L86 defines `fpSet` as the collection of valid hashed fraud-proof proposition-bytes from R4 of the `fpData` data-input.
 - L87 defines `validFP` checking that there exists a fraud-proof hash in `fpSet` that matches the blake2b256 hash of the fraud-proof script context-variable.
 - L91 defines `contractSame` checking that the output evaluation box has the same contract as the input evaluation box.
 - L91 defines `correctInputs` checking that there are only two inputs in the transaction.
 - L93 defines `tokensConserved` checking that the output evaluation box contains the same tokens as the input evaluation box.
 - L94-L102 defines `flagsConserved` checking all of the following properties of the AVL tree in the output evaluation box are true:
 - Insertions, updates, and deletions are enabled.
 - The key length is the same as the input AVL tree key length.
 - The value length option is the same as the input AVL tree value length option.
 - L107 defines `removedScore` as the difference between the `totalScore` and `nextScore` representing the score of the NISP being evaluated.
 - L108 defines `slashedBond` as the bond value calculated by applying the `bondForScore` function on the `removedScore` input.
 - L113 defines `proverInput` as the second input at index 1.
 - L114 defines `rewardOutput` as the second output at index 1.
 - L115-L120 defines `proverPaid` checking that:

4EYES Labs Audit - Lithos: Mining (Part 3)

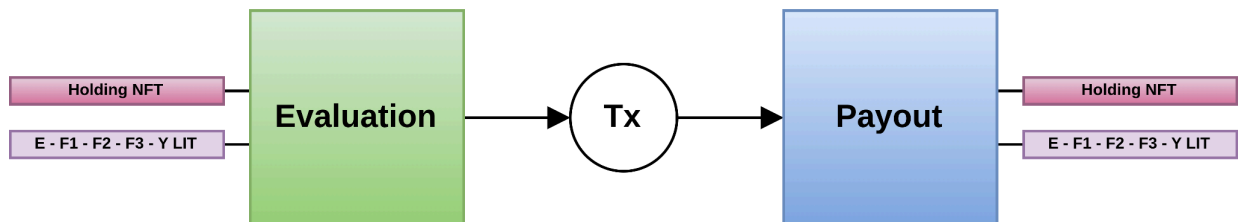
- The reward output proposition bytes equals the prover input proposition bytes.
 - The reward output equals the slashedBond value.
- L121 defines valueSlashed checking that the output evaluation box value equals the input evaluation box value minus the slashedBond value. Thus, the bond the prover gets comes from the fraudulent miner.
- L122 defines bondSlashed checking that the total bond value stored in the output evaluation box is updated by removing the calculated slashedBond value.
- L128 defines bondSolvent checking that the total bond value stored in the output evaluation box is greater than or equal to 0. This ensures that no score corresponds to a larger bond than posted.
- L130 defines samePeriod checking that the period start height is preserved in the output evaluation box.
- L131 defines sameBlock checking that the creation-height of the holding box is preserved in the output evaluation box.
- L133-L150 validSlash checking that:
 - validFP, authenticFPData, onlyOne, contractSame, correctInputs, tokensConserved, flagsConserved, stateSized, valueSlashed, bondSlashed, bondSolvent, samePeriod, sameBlock, proverPaid.
- L151 checks validSlash is true and the fraud-proof script context-variable at index 0 evaluates to true.
- L158 otherwise, the evaluation phase is over and the evaluation box becomes the payout box where:
 - L160 defines passedEvalPeriod checking that the height of the current block being validated is greater than or equal to the blockheight corresponding to the end of the evaluation phase.
 - L163 defines LITReward as the tokens at index 1 of the input evaluation box.
 - L165 defines baseReward as the value of the input evaluation box minus the total bond value.
 - L166 defines conservedValues checking that:
 - The ERG value, the tokens, the NISP AVL tree, the miner count, and the total share score.
 - The rollup state collection size remains 3.
 - The period becomes the ERG block reward value as record keeping.
 - The creation-height of the holding box becomes the LIT block reward value as record keeping.
 - The total bond value remains to be refunded by the payout box.
 - L180 defines isPayoutContract checking that the blake2b256 hash of the payout box equals the CONST_PAYOUT_PROPBYTES compile-time constant.
 - L182 checks that isPayoutContract, conservedValues, passedEvalPeriod, and onlyOne are true.

Transaction Diagrams

Evaluation: Fraud-Proof Validation



Evaluation: Convert to Payout



4EYES Labs Audit - Lithos: Mining (Part 3)

Review Comments

- The bond introduced in the holding contract for the submission phase is slashed and given to fraud-provers if valid fraud is found in the miner NISP, otherwise it will be refunded by the payout box.

Payout

Description

- Contains the ERG and LIT to be paid-out to miners of the Lithos decentralized mining pool. If no miners are in the NISP tree, then anyone can spend the box as long as there is no miner fee output box. If there is any remaining ERG in the payout box, then it will be distributed along with any remaining LIT tokens in a subsequent transaction.

Box Contents

Registers

- R4 [AvlTree]: The NISP tree, initially an empty AVL tree mapping a miner's hashed proposition bytes to their corresponding NISP.
- R5 [Int]: Total miners in a given period, representing miners who have posted a NISP.
- R6 [BigInt]: The total share score, which increases for every NISP posted. The Share Score of a NISP is the first 8 bytes of the NISP. Individual Share Scores have type Long.
- R7 [Coll[Long]]: Collection of values representing the rollout state.
 - Index 0: Total block reward.
 - Index 1: Total LIT reward.
 - Index 2: Bond total remaining.
- R8: Reserved for extensions.
- R9: Reserved for extensions.

Tokens

- Index 0: The holding box NFT that uniquely identifies this box.
- Index 1: LIT tokens to be emitted to miners of the pool.

Context Variables

- Index 0 [Coll[Coll[Byte]]]: Data representing the hashed proposition bytes of miners to retrieve from the NISP AVL Tree.
- Index 1 [Coll[Byte]]: The lookup proof for the AVL tree.
- Index 2 [Coll[Byte]]: The remove proof for the AVL tree.

Compile-Time Constants

- CONST_FEE_HASH [Coll[Byte]]: The hashed proposition bytes of the fee contract.
- CONST_MIN_ENTRY_BOUND [Long]: Floor on the refundable bond a submission is required to provide.
- CONST_BOND_DIVISOR [Long]: Score units per nanoERG of bond.

Contract Analysis

- L50 defines the `bondForScore` function taking a `bondScore` Long input and doing the following:
 - L93 defines `bondProportional` as the quotient of `bondScore` and `CONST_BOND_DIVISOR`, representing the amount of ERG required for the bond based on its score.
 - L93 checks that:
 - If the `bondProportional` value is greater than `CONST_MIN_ENTRY_BOND` then use the `bondProportional` value.
 - Otherwise, use the `CONST_MIN_ENTRY_BOND` value.
- L55 defines `numTokenInOutOutputs` as a method taking a `Coll[Byte]` as input representing a token id and returning the amount of that token in each output box, if it exists.
- L69 If the amount of miners is 0, either no miners submitted NISP proofs or all were removed from the AVL tree due to fraud, then the following is checked:
 - L71 defines `hasFeeOutput` checking that there exists an output box whose proposition bytes hash equals the `CONST_FEE_HASH` compile-time constant.
 - L75 defines `onlyOne` checking that the payout box is the first input at index 0.
 - L78 defines `noFeeOutput` setting `hasFeeOutput` to its negation.
 - L79 defines `nftBurned` checking that the token at index 0 of the input payout box does not exist anywhere in the outputs of the transaction.
 - L81 defines `noBondliability` checking that the total bond amount equals 0 (i.e. since all miners were fraudulent, provers claimed their NISP bond).
 - L82 checks that `noFeeOutput`, `onlyOne`, `nftBurned`, and `noBondLiability` are true.
- L83 Otherwise, miners of the pool who were not removed from the NISP AVL tree due to fraud get rewarded accordingly:
 - L85 defines `minersToPay` as a collection of tuples containing the miner hashed proposition bytes and their NISP, the data coming from the `CTX_KEY_DATA` context-variable.
 - L88 defines `indexedKeys` zipping the miner data with the data indices.
 - L89 defines `nonUniqueKeys` checking that there exists at least one pair of identical keys in the data.
 - L95 If for each miner entry, the corresponding NISP proof size is 40, then:
 - L96 defines `minersWithScore` as the collection `minersToPay` where only the first 8 bytes from the NISP proof are converted to a Long representing the share score.
 - L103 defines `minersWithBond` as the collection `minersWithScore` where the share score is replaced with the calculated bond amount for that score using the `bondForScore` method.
 - L110 defines `minersWithReward` as the collection `minersWithScore` where the share score is replaced with the proportional amount of ERG rewards based on the share score plus their bond refund calculated from the score.

4EYES Labs Audit - Lithos: Mining (Part 3)

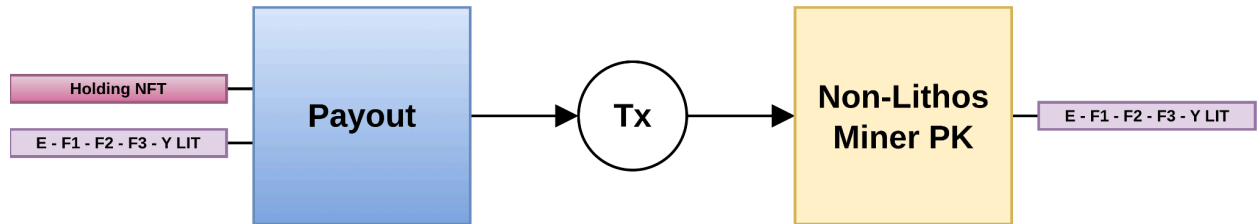
- L119 defines minersWithLIT as the collection minersWithScore where the share score is replaced with the proportional amount of LIT rewards based on the share score.
- L125 defines totalMinerRewards as the sum of the calculated proportional miner ERG rewards from minersWithReward.
- L130 defines totalLITRewards as the sum of the calculated proportional miner LIT reward from minersWithLIT.
- L135 defines totalBondRefunds as the sum of the calculated miner bonds from the minersWithBond.
- L140 defines remainder as the difference between the input payout box value and totalMienerRewards.
- L143 if totalMinerRewards is less than the input payout box value and remainder is greater than the minimum box value of 1_000_000 nanoERG, then:
 - L156 defines minerOutputBoxes as the slice of the outputs after the payout box representing the miners getting paid.
 - L162 allMinersPaid checking that the minerOutputBoxes slice has size equal to minersWithReward (i.e. so even though the slice was meant to reach up until minersWithReward.size + 1, there are not actually fewer outputs in the slice).
 - L163 defines treeUpdated checking that miners in CTX_KEY_DATA are removed from the NISP tree and that this new tree digest is in the output payout box.
 - L167 defines minersRewarded checking that all miners in the output slice minerOutputBoxes have the corresponding proposition bytes and value from the minersWithReward collection.
 - L171 defines LITRewarded checking that:
 - If the LIT token reward is greater than 0, then:
 - For all miners in the output slice minerOutputBoxes, check that if the calculated miner token reward is positive, then check that the correct token id and token amount exist at token index 0 of the miner output.
 - Otherwise, return true.
 - Otherwise, return true.
 - L186 defines validUTXO checking that the output payout box has the same proposition bytes as the input payout box.
 - L187 defines stateSized checking that the rollup state collection is maintained to equal 3.
 - L188 defines usedReward checking that the output payout box value equals the remainder amount.

4EYES Labs Audit - Lithos: Mining (Part 3)

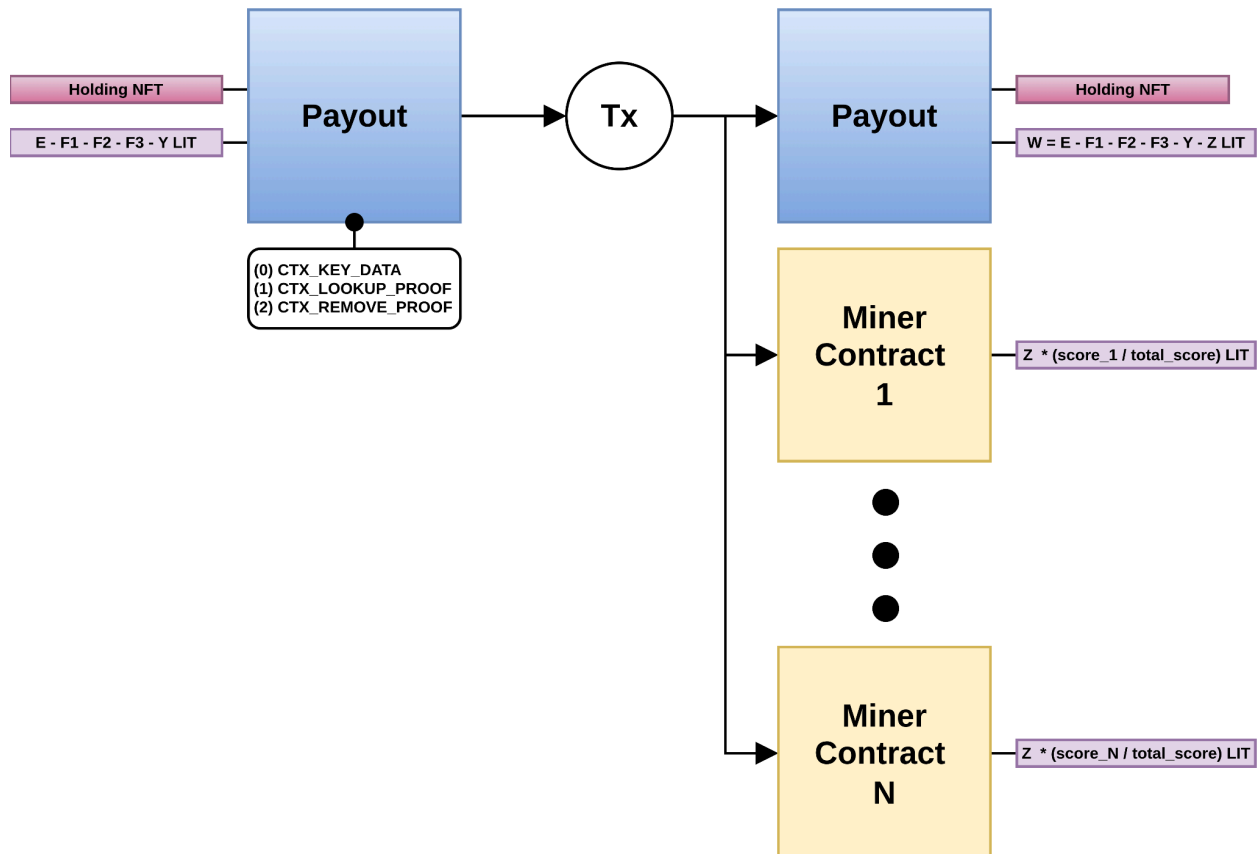
- L189 defines usedLIT checking that the output payout box contains the LIT token id and the amount in the input payout box minus the calculated totalLITRewards.
- L192 defines usedBond checking that the total bond value in the output payout box is updated to account for the refunded bonds removed from the input.
- L193-L196 checks that the output payout box keeps the same total reward, total score, miner count, and token reward as the input payout box.
- L197 defines onlyOne checking that the input payout box is the first input at index 0.
- L198 defines statements checking that all previously stated conditions are true, that the rollup NFT in the output payout box equals the token in the input payout box, and that nonUniqueKeys is false.
- L218 checks that statements is true.
- L219 otherwise, there is no remaining ERG in the box and the payout box is spent where:
 - L220 defines minerOutputBoxes as the slice of the outputs representing the miners getting paid. There is no payout output box.
 - L227 defines allMinersPaid checking that the minerOutputBoxes slice has size equal to minersWithReward (i.e. so even though the slice was meant to reach up until minersWithReward.size + 1, there are not actually fewer outputs in the slice).
 - L228 defines minersRewarded checking that all miners in the output slice minerOutputBoxes have the corresponding proposition bytes and value from the minersWithReward collection. The inequality allows for ensuring that any residual ERG can be added to any one of the miner outputs.
 - L233 defines LITRewarded in the same way as in the previous spending path.
 - L248 defines onlyOne checking that the input payout box is the first input at index 0.
 - L249 defines nftBurned checking that the rollup NFT does not exist anywhere in the outputs.
 - L251 defines allBondsRefunded checking that the cumulative totalBondRefunds equals the total bond value stored in the input payout box.
 - L253 defines statements checking that all previously stated conditions are true and that nonUniqueKeys is false.
 - L264 checks that statements is true.
- L266 Otherwise, evaluate to false.

Transaction Diagrams

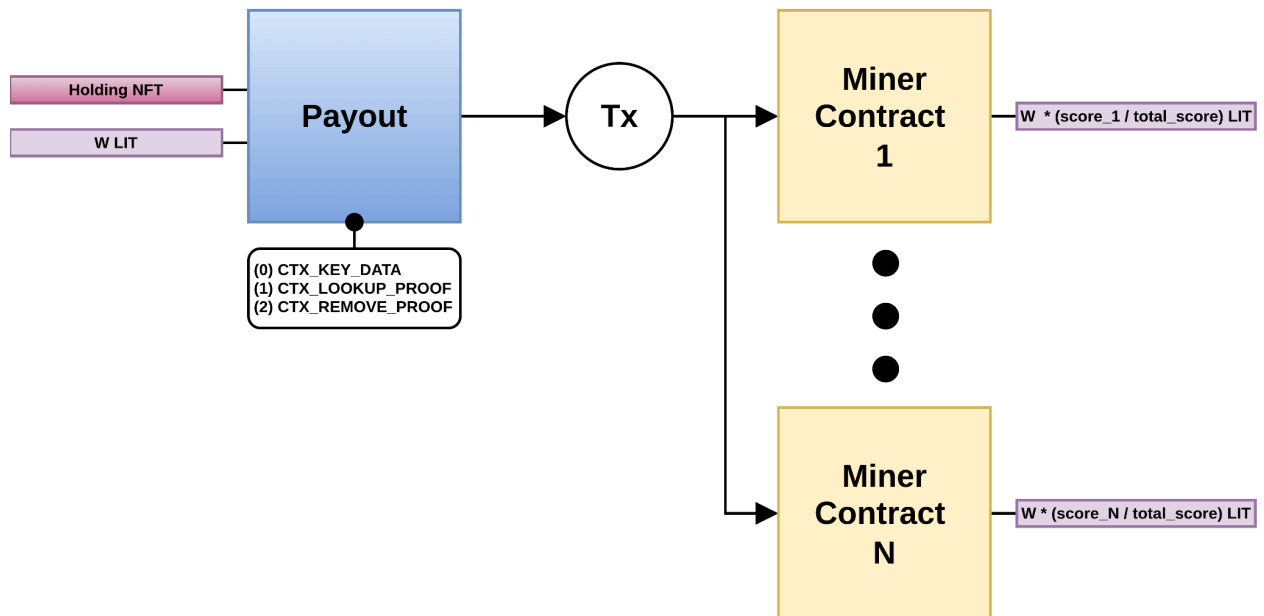
Payout: No NISP Miners



Payout: Leftover ERG



Payout: No Leftover ERG



Review Comments

- Along with the proportional ERG and LIT rewards, the miners get their bond refunded.

Appendix

Issue Guide

Severity	Description
CRITICAL	The contract contains one or more serious errors and warnings.
SERIOUS	An error that will cause unexpected behavior.
WARNING	A property that is questionable but will not cause unexpected behavior.
GOOD	Everything works as expected.

Status	Description
RESOLVED	The issues have been addressed. Default status if severity is GOOD.
UNRESOLVED	The issues have not been addressed.

About Us

4EYES Labs is an ErgoScript design and auditing company for Ergo blockchain.

Linktree: <https://linktr.ee/4EYESLabs>